

Zserio SIG Agenda

- The most important problems for C++17 generator:

#91	Improve Access to View and Array Types	In Clarification
#90	Amalgamation Support is Missing	Pending
#92	Debug builds with a huge choice are very demanding on memory	Done
#89	Emitted c++ code for templates should have a using declaration per template argument	Pending
#88	Optional with default value is not reset during read	Done
#79	Wrong code generated for choice with only default case	Done
#93	Adapt runtime to build as shared library	In Progress
#98	Client Methods do not Respect -withTypeInfoCode	Pending

- If you have something more, please don't hesitate to contact us

#91: Data-View Design

- We have 5 different ideas how to redesign Data View abstractions
- Very quick recapitulation:
 - 1. **Namespace** approach – Top level namespace with inner Data and View classes
 - 2. **Using** approach – Same as now but with inner Data and View usings
 - 3. **Top namespace** approach – Data and View top level namespace
 - 4. **Top View** approach – Only View top level namespace
 - 5. **Global Data View** approach – Global Data and View namespace
- All of them have some drawbacks

- Current approach – Data class and View class in the namespace

```
struct Field<T>
{
    T value;
    T arr[3];
};

template <typename T>
struct Field
{
    T value;
    ::zserio::Vector<T> arr;
};

template <typename T>
class View<Field<T>>
{
};
```

- Data and View classes are generated in the namespace given by the package in the schema

- **Namespace** approach – Top level namespace with inner Data and View classes

```
struct Field<T>
{
    T value;
    T arr[3];
};

namespace Field
{
    template <typename T>
    struct Data
    {
        T value;
        ::zserio::Vector<T> arr;
    };

    template <typename T>
    using View = ::zserio::View<Data<T>>; // just trick for now
};
```

#91: Data-View Design

- **Namespace** approach – Top level namespace with inner Data and View classes
 - *Noisy syntax (Data will be everywhere):*
`Field::Data<zserio::Int32> data;`
 - *Template parameters will be at the end and not after Field*
 - *Namespace Field will be named with a capital F to avoid awkward clashes*
 - Note:
 - Some NDS users already claimed that they are interested only in the Data part
 - They will be probably not very happy with extra 'Data' syntax

- **Using** approach – Same as now but with inner Data and View usings

```
struct Field<T>
{
    T value;
    T arr[3];
};

template <typename T>
struct Field
{
    using Data = Field;
    using View = ::zserio::View<Field>;

    T value;
    ::zserio::Vector<T> arr;
};

template <typename T>
class View<Field<T>>
{
};
```

#91: Data-View Design

- **Using** approach – Same as now but with inner Data and View usings

- *There will be two different possibilities how to use Data and View:*

```
Field<zserio::Int32> data;  
zserio::View<Field<zserio::Int32>> view;
```

- or

```
Field<zserio::Int32>:Data data;  
Field<zserio::Int32>:View view;
```

- *There will be a clash if Data or View keywords will be used in the schema:*

```
struct Data<T>  
{  
    T value;  
    T arr[3];  
};
```


#91: Data-View Design

- **Top namespace** approach – Data and View top level namespace

```
struct Field<T>
{
    T value;
    T arr[3];
};

namespace Data
{
    template <typename T>
    struct Field
    {
        T value;
        ::zserio::Vector<T> arr;
    };
}

namespace View
{
    template <typename T>
    using Field = ::zserio::View<Field<T>>; // hack for now
}
```

#91: Data-View Design

- **Top namespace** approach – Data and View top level namespace
 - *It does not look intuitive because it does not reflect the Zserio schema*
 - *NDS users who are interested only to Data part will be probably not very happy with this solution*

- **Top View** approach – Only View top level namespace

```
struct Field<T>
{
    T value;
    T arr[3];
};

template <typename T>
struct Field
{
    T value;
    ::zserio::Vector<T> arr;
};

namespace View
{
    template <typename T>
    using Field = ::zserio::View<Field<T>>; // hack for now
}
```

#91: Data-View Design

- **Top View** approach – Only View top level namespace
 - *It does not look intuitive because it does not reflect the Zserio schema*

#91: Data-View Design

- **Global Data View** approach – Global Data and View namespace

```
struct Field<T>
{
    T value;
    T arr[3];
};

namespace Data
{
    namespace ZserioPackage
    {
        template <typename T>
        struct Field
        {
            T value;
            ::zserio::Vector<T> arr;
        };
    }
}

namespace View
{
    namespace ZserioPackage
    {
        template <typename T>
        using Field = ::zserio::View<Field<T>>; // hack for now
    }
}
```

#91: Data-View Design

- **Global Data View** approach – Global Data and View namespace
 - *It does not look intuitive because it does not reflect the Zserio schema*

- Very quick summary from our side:
 - All of them result in a slightly non-intuitive change to the current generated design
 - The only reason for this change would be to ease the porting from the C++11 generator
 - But what about newcomers?
 - While they won't have any problems with the porting, they will likely start to complain about the cumbersome generated code and they would be right
 - *Our recommendation, despite potential alternatives, is to proceed without changes*

#92: Variant Problem

- Variant was useless for the extreme number of template arguments (>300)
- **Each compiler** had a severe problem to deal with such a variant
- Because NDS.Classic uses a choice with more than 300 cases and the C++17 generator should be generic, we had to find out some solution
- The solution what we found was to replace `std::variant` by `zserio::Any` in our `zserio::Variant` implementation
- `zserio::Variant` interface has not been changed
- Compilation is working now except for clang compiler which requires option `"-fbracket-depth=1024"`

#90: Amalgamation Support

- Amalgamation support can be implemented, of course
- Our intention is to keep C++17 command line arguments as simple as possible
- We suggest to measure improvements before the implementation
- Some numbers for NDS.Classic 2.5.4 compilation:
 - C++11 without amalgamation took 2:09 minutes
 - C++11 with amalgamation took 1:24 minutes
 - C++17 without amalgamation took 4:00 minutes
- It seems that the main problem is the extensive use of templates